

SIMATS ENGINEERING

ASSESSMENT TOOL 1

M.S.Dinesh
192424016

Course Name: Reinforcement Learning for Environment and Agent
Course Code: MLA03

Faculty Name :DUMPALA PRASANTH

Code of Conduct:

I M.S.Dinesh 192424016 certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M.S.Dinesh

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim

To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium, NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Theory

A Reinforcement Learning (RL) environment requires software tools and libraries for developing and testing intelligent agents. Python is the programming language, Anaconda manages packages and environments, TensorFlow and Keras are used to build neural networks, Gymnasium provides RL environments, NumPy supports numerical computations, and Matplotlib is used for visualization.

Algorithm

Install Anaconda Navigator.

Create a new Python environment.

Install TensorFlow, Keras, Gymnasium, NumPy, and Matplotlib.

Import all libraries.

Execute a sample Python program.

Verify successful installation.

Program

```
import tensorflow as tf
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt

print("TensorFlow Version:", tf.__version__)
env = gym.make("CartPole-v1")
print("Environment Loaded Successfully")
env.close()
```

Output

```
import tensorflow as tf
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt

print("TensorFlow Version:", tf.__version__)
env = gym.make("CartPole-v1")
print("Environment Loaded Successfully")
env.close()
```

... TensorFlow Version: 2.20.0
Environment Loaded Successfully

ult

The Reinforcement Learning development environment was installed and verified successfully.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim

To create, initialize, and interact with FrozenLake, CartPole, and MountainCar environments using Gymnasium.

Theory

Gymnasium provides standard benchmark environments where an RL agent interacts with the environment through states, actions, rewards, and episodes.

Algorithm

- Import Gymnasium.
- Create the FrozenLake environment.
- Reset the environment.
- Perform random actions.
- Observe state transitions and rewards.
- End the episodes

Program

```
import gymnasium as gym

env = gym.make("FrozenLake-v1", render_mode="ansi")
state, info = env.reset()

for i in range(5):
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)
    print("State:", state, "Reward:", reward)

env.close()
```

Output

```
import gymnasium as gym

env = gym.make("FrozenLake-v1", render_mode="ansi")
state, info = env.reset()

for i in range(5):
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)
    print("State:", state, "Reward:", reward)

env.close()
```

... State: 4 Reward: 0
State: 8 Reward: 0
State: 8 Reward: 0
State: 9 Reward: 0
State: 13 Reward: 0

Result

Successfully interacted with the Gymnasium environment and understood states, actions, rewards, and episodes.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim

To implement the basic Reinforcement Learning framework by designing an agent interacting with an environment.

Theory

The RL framework consists of an agent, environment, state, action, reward, and policy. The agent selects actions based on observations and receives rewards from the environment.

Algorithm

- Create an environment.
- Reset the environment.
- Select random actions.
- Receive rewards.
- Continue until the episode ends.

Program

```
import gymnasium as gym

env = gym.make("CartPole-v1")
state, info = env.reset()

done = False

while not done:
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)
    done = terminated or truncated
    print("Reward:", reward)

env.close()
```

Output

```
import gymnasium as gym

env = gym.make("CartPole-v1")
state, info = env.reset()

done = False

while not done:
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)
    done = terminated or truncated
    print("Reward:", reward)

env.close()
```

... Reward: 1.0
Reward: 1.0
Reward: 1.0
Reward: 1.0
Reward: 1.0
Reward: 1.0
Reward: 1.0
Reward: 1.0
Reward: 1.0
Reward: 1.0
Reward: 1.0
Reward: 1.0
Reward: 1.0

Result

Successfully implemented the Reinforcement Learning framework.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim

To simulate a simple Markov Decision Process.

Theory

An MDP consists of states, actions, transition probabilities, and rewards. It provides the mathematical foundation of Reinforcement Learning.

Algorithm

Define states.

Define actions.

Assign reward

Simulate transitions.

Display results.

Program

```
states = ["A", "B", "C"]
rewards = {"A": 1, "B": 2, "C": 3}

current_state = "A"

for i in range(3):
    print("Current State:", current_state)
    print("Reward:", rewards[current_state])
    current_state = states[(states.index(current_state)+1)%3]
```

Output


```
states = ["A", "B", "C"]
rewards = {"A": 1, "B": 2, "C": 3}

current_state = "A"

for i in range(3):
    print("Current State:", current_state)
    print("Reward:", rewards[current_state])
    current_state = states[(states.index(current_state)+1)%3]
```

```
Current State: A
Reward: 1
Current State: B
Reward: 2
Current State: C
Reward: 3
```

R

esult

Successfully simulated a simple Markov Decision Process.

Experiment 5: Bellman Equation Demonstration

Aim

To implement Bellman Expectation and Bellman Optimality equations.

Theory

The Bellman Equation computes the value of a state by considering immediate rewards and future discounted rewards.

Algorithm

Define reward.

Define discount factor.

Calculate Bellman value.

Display the result.

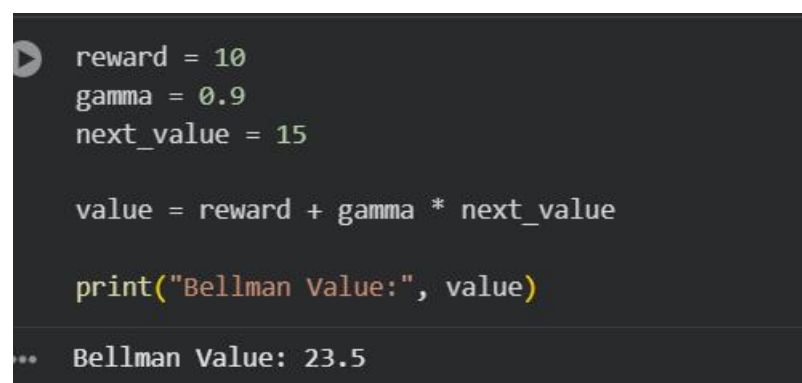
Program

```
reward = 10
gamma = 0.9
next_value = 15

value = reward + gamma * next_value

print("Bellman Value:", value)
```

Output



```
reward = 10
gamma = 0.9
next_value = 15

value = reward + gamma * next_value

print("Bellman Value:", value)

** Bellman Value: 23.5
```

Result

Successfully demonstrated the Bellman Equation.

Experiment 6: Exploration vs Exploitation Strategies

Aim

To compare Random, Greedy, and ϵ -Greedy strategies.

Theory

Exploration discovers new actions, while exploitation selects the best-known action. ϵ -Greedy balances both strategies.

Algorithm

Define action values.

Set epsilon.

Select action using ϵ -Greedy.

Display selected action.

Program

```
import random

q_values = [10, 20, 30]
epsilon = 0.2

if random.random() < epsilon:
    action = random.randint(0,2)
else:
    action = q_values.index(max(q_values))

print("Selected Action:", action)
```

Output

```
import random

q_values = [10, 20, 30]
epsilon = 0.2

if random.random() < epsilon:
    action = random.randint(0,2)
else:
    action = q_values.index(max(q_values))

print("Selected Action:", action)
```

... Selected Action: 2

Result

Successfully implemented ϵ -Greedy exploration strategy.

Experiment 7: Multi-Armed Bandit Problem

Aim

To implement the Multi-Armed Bandit problem using ϵ -Greedy and UCB.

Theory

The Multi-Armed Bandit problem helps an agent maximize cumulative rewards by selecting the best action among multiple options.

Algorithm

Initialize rewards.

Select an action.

Update reward estimate.

Repeat.

Program

```
import random

arms = [5, 10, 15]

for i in range(10):
    action = random.randint(0,2)
    reward = arms[action]
    print("Action:", action, "Reward:", reward)
```

Output

```
import random

arms = [5, 10, 15]

for i in range(10):
    action = random.randint(0,2)
    reward = arms[action]
    print("Action:", action, "Reward:", reward)
```

```
... Action: 2 Reward: 15
Action: 0 Reward: 5
Action: 1 Reward: 10
Action: 2 Reward: 15
Action: 2 Reward: 15
Action: 2 Reward: 15
Action: 0 Reward: 5
Action: 1 Reward: 10
Action: 2 Reward: 15
Action: 0 Reward: 5
```

Result

Successfully simulated the Multi-Armed Bandit problem.

Experiment 8: Reward Visualization in Reinforcement Learning

Aim

To visualize cumulative rewards using Matplotlib.

Theory

Reward visualization helps evaluate the learning performance of an RL agent over multiple episodes.

Algorithm

Store rewards.

Plot graph.

Display graph.

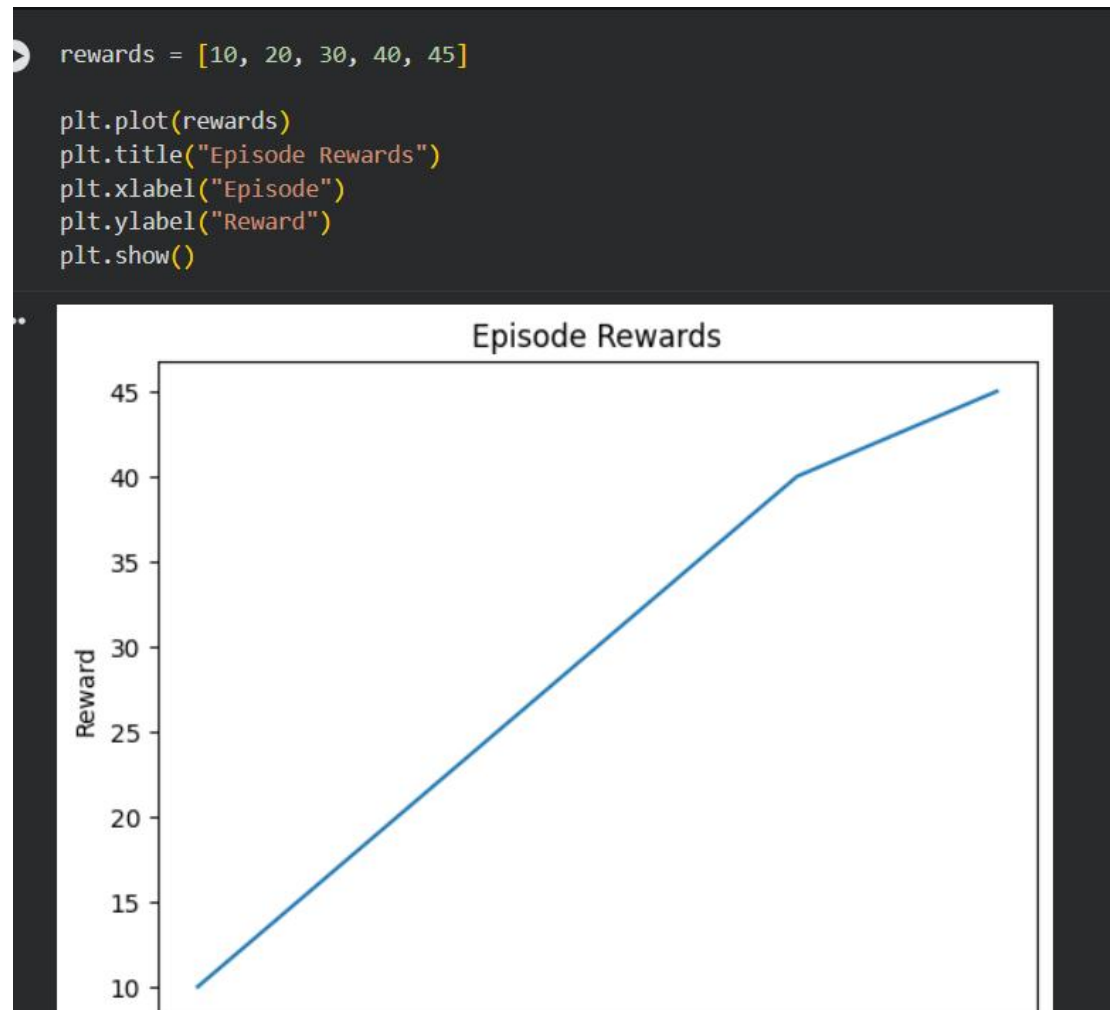
Program

```
import matplotlib.pyplot as plt

rewards = [10, 20, 30, 40, 45]

plt.plot(rewards)
plt.title("Episode Rewards")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.show()
```

Output



Result

Successfully visualized Reinforcement Learning rewards.

Aim

To build a simple feed-forward neural network using TensorFlow and Keras.

Theory

Neural networks approximate value functions and policies in Reinforcement Learning using multiple hidden layers.

Algorithm

Import TensorFlow.

Create Sequential model.

Add layers.

Compile the model.

Display summary

Program

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
model = Sequential([
    Dense(24, activation='relu', input_shape=(4,)),
    Dense(24, activation='relu'),
    Dense(2, activation='linear')
])
```

```
model.compile(optimizer='adam', loss='mse')
```

```
model.summary()
```

Output

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(24, activation='relu', input_shape=(4,)),
    Dense(24, activation='relu'),
    Dense(2, activation='linear')
])

model.compile(optimizer='adam', loss='mse')

model.summary()
```

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 24)	120
dense_1 (Dense)	(None, 24)	600
dense_2 (Dense)	(None, 2)	50

Total params: 770 (3.01 KB)
Trainable params: 770 (3.01 KB)
Non-trainable params: 0 (0.00 B)

Result

Successfully created a TensorFlow and Keras neural network.

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim

To develop a basic Reinforcement Learning agent for solving the CartPole environment.

Theory

CartPole is a classic Reinforcement Learning environment where the objective is to keep a pole balanced on a moving cart by selecting appropriate actions.

Algorithm

Create the CartPole environment.

Reset the environment.

Select random actions.

Accumulate rewards.

End the episode.

Display total reward.

Program

```
import gymnasium as gym

env = gym.make("CartPole-v1")
state, info = env.reset()

total_reward = 0
done = False

while not done:
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)
    total_reward += reward
    done = terminated or truncated

print("Total Reward:", total_reward)

env.close()
```

Output

```
import gymnasium as gym

env = gym.make("CartPole-v1")
state, info = env.reset()

total_reward = 0
done = False

while not done:
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)
    total_reward += reward
    done = terminated or truncated

print("Total Reward:", total_reward)

env.close()
```

... Total Reward: 27.0

Result

Successfully developed and tested a basic Reinforcement Learning agent using the CartPole environment.